NF-HEDM scripts: refactor, spatially adaptive reconstruction, orientation search, refinement and grain segmentation

`scripts/hedm_oct2026`, branch `nf-hedm-refactor-and-algorithms`

October 2026

Contents

1 Summary **2**

2 Branches **2**

3 Refactor **3**

3.1 `nfutil` as a package . . . 3

3.2 The `Experiment` class . . . 3

3.3 Chunking . . . 3

3.4 Orientation refinement re-centers as intended . . . 3

4 Spatially adaptive reconstruction **3**

5 Orientation-space search for missing grains **5**

5.1 How the search works . . . 5

5.2 Accuracy and speed . . . 6

5.3 Leave-out test . . . 6

5.4 `find_missing_grains.py` . . . 7

5.5 Dropping orientations that are already known . . . 7

6 Faster orientation refinement, identical results **8**

7 Refinement along the marching **9**

7.1 The boundary pass . . . 11

8 Grain segmentation and misorientation from the grain average **13**

9 Tests **16**

10 Limitations and open items **17**

1 Summary

This report describes the work on the NF-HEDM scripts in `scripts/hedm_oct2026`, starting from the single-file `nfutil.py` and the scripts that call it. The work has two halves.

- **Refactor.** `nfutil.py` became a package, the scripts share an `Experiment` class, the chunking code was written once, and the orientation refinement now re-centers as it was meant to. A regression test on Ti-7Al data guards the results.
- **New methods.** A spatially adaptive reconstruction that tests each voxel against its neighbors' grains first; an orientation-space search for grains missing from the far-field (FF) list; a faster orientation refinement that gives identical results; refinement along the marching with a boundary pass; and grain segmentation with the misorientation of every voxel from its grain's average.

All numbers are for Ti-7Al (hexagonal, 619 FF grains for the initial scan and 546 for the final scan), one layer of 39,762 voxels, on 9 CPUs, unless said otherwise. The initial scan uses the tomography mask, and the final scan uses the initial scan's mask (the two are almost identical). Everything was validated on this one material, so other materials and symmetries should be checked before the settings are trusted.

Change	Result on Ti-7Al
Spatially adaptive reconstruction	38% of the orientation tests, about half the time, 99% of voxels on the same grain
Orientation-space search	finds orientations missing from the FF list; 5 of 6 removed grains recovered to within 0.15°
Faster refinement	0.04 s per voxel instead of 0.24 s, identical orientations and confidences
Refinement along the marching	same confidence as refining each voxel alone, 3.0 to 3.5 times faster
Boundary pass	agrees with refining each voxel alone at 99.85% and 99.97% of voxels (92.9% and 93.5% without)
Grain segmentation	grains from the refined field; median misorientation from the grain average 0.02°

Table 1: The main results. Details and the figures behind them follow.

2 Branches

The work was developed as a stack of branches of `rachelelim/hexrd-1`, each building on the one before. The stack is now consolidated in one branch, `nf-hedm-refactor-and-algorithms`, which contains all of it; the earlier branches (except the merged package split) were deleted after their commits were confirmed to be in it. The table shows the order the work was done in.

Branch	Contents
`nfutil-package-split`	package split, `Experiment` class, chunking cleanup, re-centering fix (merged)
`spatial-marching-reconstruction`	spatially adaptive reconstruction, Ti-7Al regression tests
`orientation-space-search`	orientation-space search, reworked `find_missing_grains.py`
`nf-hedm-refactor-and-algorithms` (was `orientation-refinement`)	faster refinement, refinement along the marching, boundary pass, segmentation, `known_tol_deg`

3 Refactor

3.1 `nfutil` as a package

`nfutil.py` was split into a package. The public names are re-exported from `nfutil/__init__.py`, so `import nfutil` and `nfutil.function(...)` work as before.

Module	Holds
`experiment.py`	the `Experiment` class
`process_controller.py`	CPU count, chunk size and multiprocessing method
`processor_functions.py`	the functions run in worker processes, refinement, chunking
`data_io.py`	reading and writing data
`graphics.py`, `utilities.py`	plotting and helpers
`spatial_adaptive.py`	marching reconstruction and refinement (new)
`orientation_search.py`	orientation-space search (new)
`simulation.py`	fast peak simulation (new)
`segmentation.py`	grain segmentation (new)

Worker functions live in `processor_functions.py` so that the multiprocessing pool can import them.

3.2 The `Experiment` class

The scripts used to repeat the same set-up: read the config, load the detector, images, material and FF grains, and build the orientation list. This is now one object, `nfutil.Experiment(cfg)`, which holds the plane data, detector parameters, omega ranges, image stack and FF orientations, and can generate test coordinates. Every script was converted to it. A single-layer regression test (now on Ti-7Al data) checks that a reconstruction still matches a stored reference.

3.3 Chunking

The code that splits voxels or orientations into chunks for the pool was repeated in several places. It is now written once as `make_chunks`, and `run_chunked` runs a distributor over the chunks and collects the results.

3.4 Orientation refinement re-centers as intended

The refinement is a pattern search: at each radius it scores a $3 \times 3 \times 3$ grid of small rotations around the current best orientation and should move to the best of them and score a new grid while the confidence improves. The re-centering did not work that way. It now re-centers on the best point while the confidence strictly improves (`_pattern_search_refine`), and tests check it on bowl-shaped confidence functions.

4 Spatially adaptive reconstruction

Reconstructing a layer tests every FF grain's orientation at every voxel (the *all-grains test*), which is the reference here. Neighboring voxels almost always belong to the same grain, so most of those tests are wasted. `reconstruct_nf.py --spatially-adaptive` (or `nfutil.reconstruct_spatially_adaptive`) marches instead:

1. Seeds on a sparse grid (every 8th voxel) are tested against every FF grain.
2. A voxel next to converged voxels (confidence ≥ 0.7) is tested against only the grains found at those neighbors, and accepted if its confidence is at least 0.8 times the best neighbor's.
3. Voxels that fail, or are cut off from the seeds, are tested against every grain and the march continues.

Only grains in the FF list can be found. On the initial scan, 2% of voxels are seeds, 62% are copied from neighbors and 37% fall back to the all-grains test (Figure 1). With the defaults the reconstruction takes 40 s against 83 s for the all-grains test on the initial scan (37 s against 73 s on the final scan), uses 38% of the orientation tests, puts 99.0% of voxels on the same grain, and 99.9% end within 0.05 of the best confidence (Figure 2). Lower converged-confidence settings are faster but lose accuracy: at 0.6 the time falls to 17% but only 94% of voxels keep the same grain.

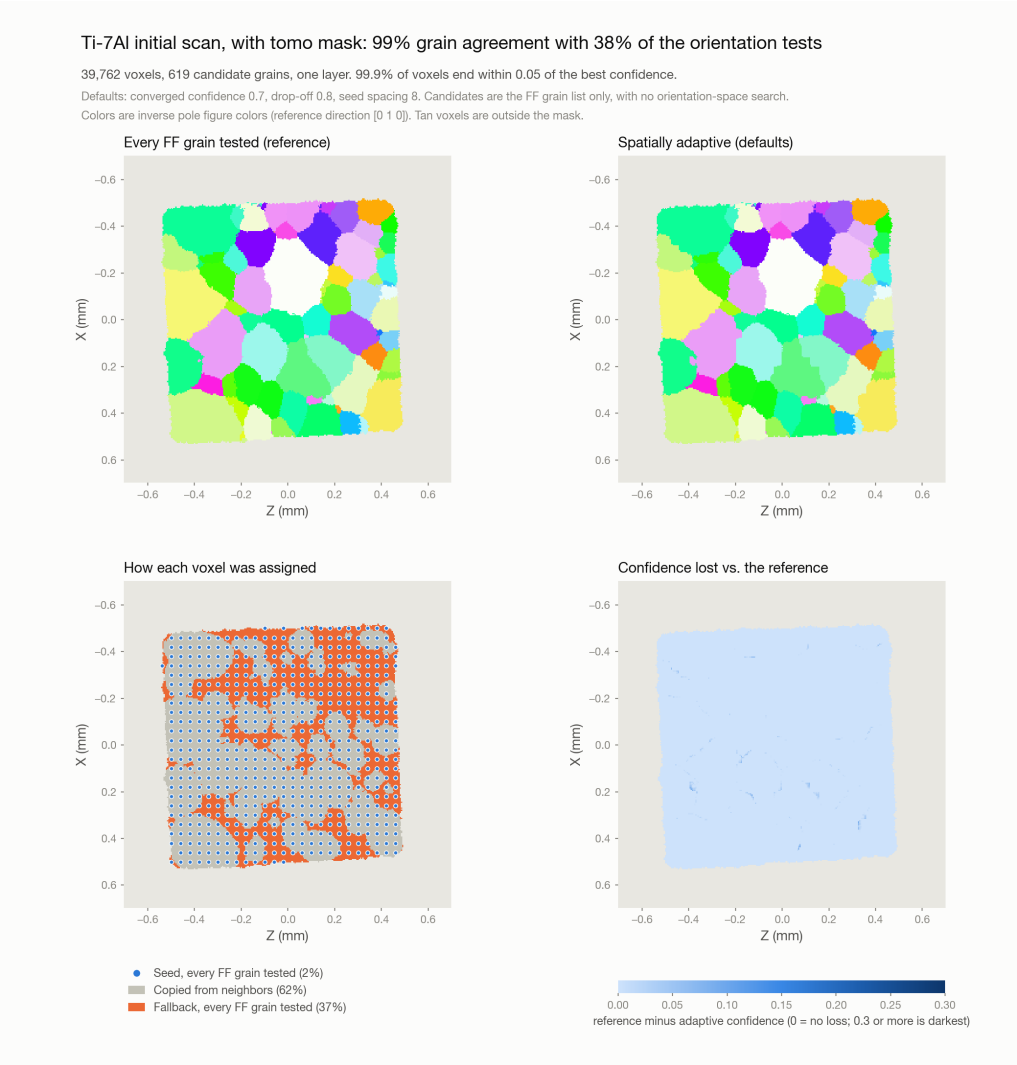


Figure 1: Initial scan with the tomography mask. The spatially adaptive reconstruction (top right) matches the all-grains reference (top left); bottom left shows how each voxel was assigned, and bottom right the confidence lost against the reference, which is almost none.

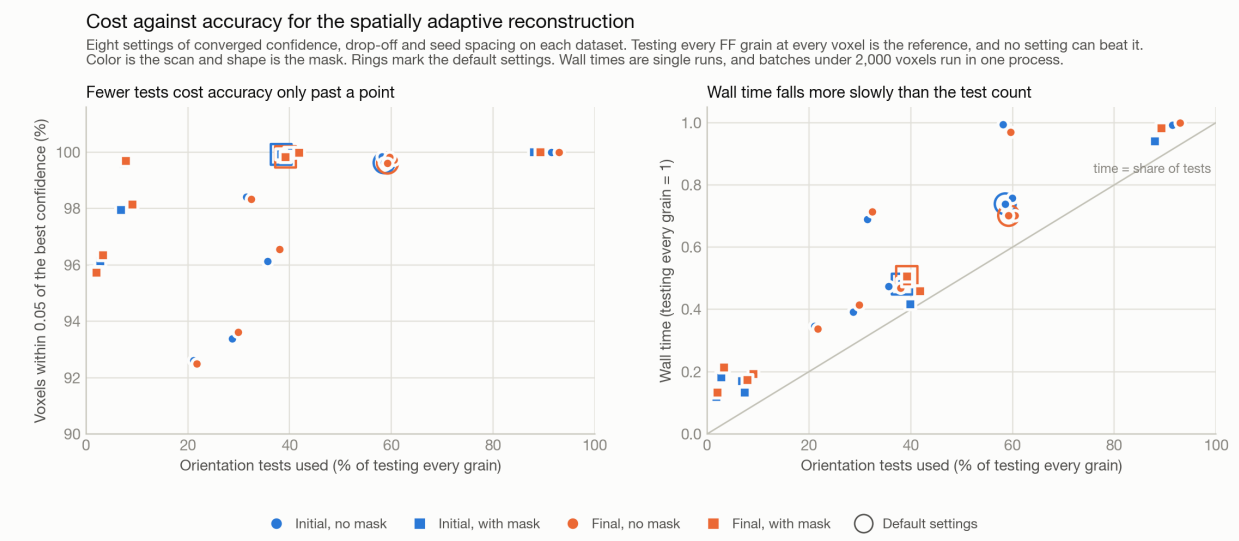


Figure 2: Cost against accuracy for eight settings on each dataset. Fewer tests cost accuracy only past a point, and wall time falls more slowly than the test count. Rings mark the defaults.

5 Orientation-space search for missing grains

Grains missing from the FF list show up in the NF reconstruction as low-confidence regions. `nfutil.search_orientations` finds the best orientation at each of a batch of voxels without a candidate list. The **confidence** of an orientation at a voxel is the fraction of its simulated Bragg peaks that land on lit pixels of the binarized images.

5.1 How the search works

1. Sample the fundamental zone of the crystal symmetry on a grid (1.75°, about 304,000 orientations for hexagonal).
2. Score every grid orientation at every voxel with the 12 lowest-angle rings and keep the best 200 per voxel.
3. Cluster those (symmetry aware) and keep the best 20 clusters. This step matters most: the right basin is often not the single highest-scoring grid point.
4. Refine each cluster with the pattern search (radii 1.5, 1.0, 0.5°) and keep the best 4.
5. Refine those with radii 0.5, 0.25, 0.1°using every ring. The best is the answer.

The design follows Li and Suter (J. Appl. Cryst., 2013) but not their coarse stage: they score a coarse grid with only the lowest-order reflections. We measured whether that works here. It does not, because fewer rings do not broaden the confidence peak, they only make it noisier (Figure 3). The first stage therefore needs a fine grid and most of the rings.

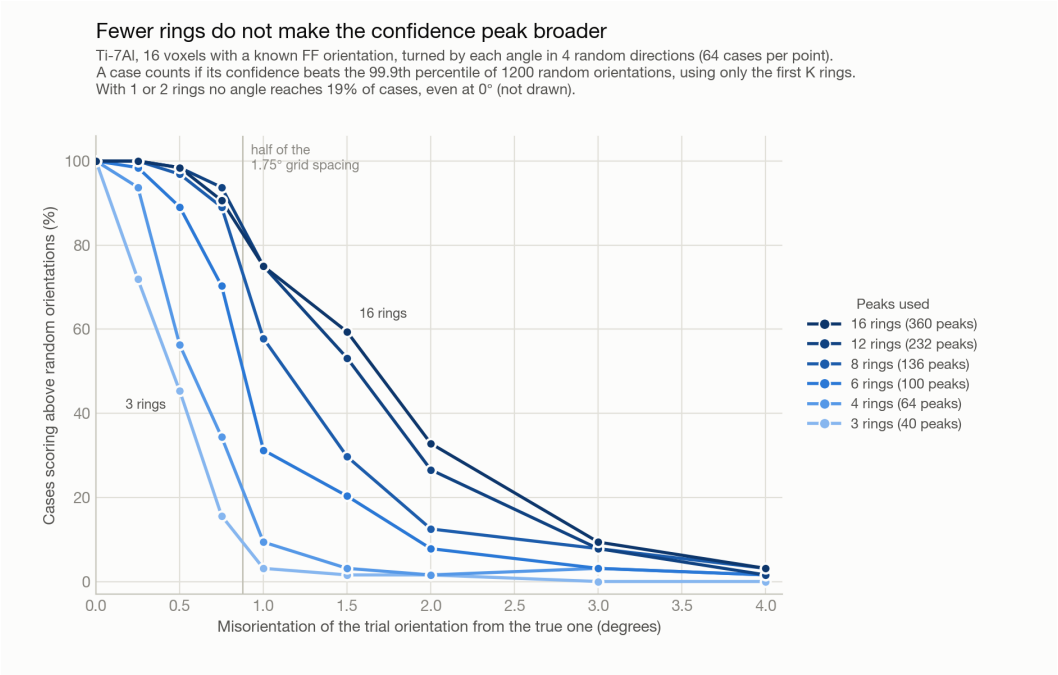


Figure 3: Share of cases whose confidence beats random orientations, against misorientation from the true orientation, for different numbers of rings. Using fewer rings does not widen the peak.

5.2 Accuracy and speed

- At 100 fresh high-confidence voxels with a known FF orientation, the search recovered it to within 0.5°at 96%. At weaker voxels (reference confidence 0.6 to 0.8) it was 62%, and 77% found an orientation about as good as the FF one.
- The original (legacy) search refines only the single best grid point. On the same 1.75°grid it recovered 72% against 93%. At its default 1°grid it recovers about as many, at roughly 25 to 30 s per voxel (an estimate) and about 74 GB of memory.
- The new search takes about 1 s per voxel plus about 13 s per batch.

5.3 Leave-out test

Six grains were removed from the FF list and the search was asked to find them again (Figure 4). It recovered five to within 0.15°. Voxels below 0.5 confidence fell from 2,504 to 547 (701 with the full list). The sixth was not found: the nearest orientation kept was 22.5°away.

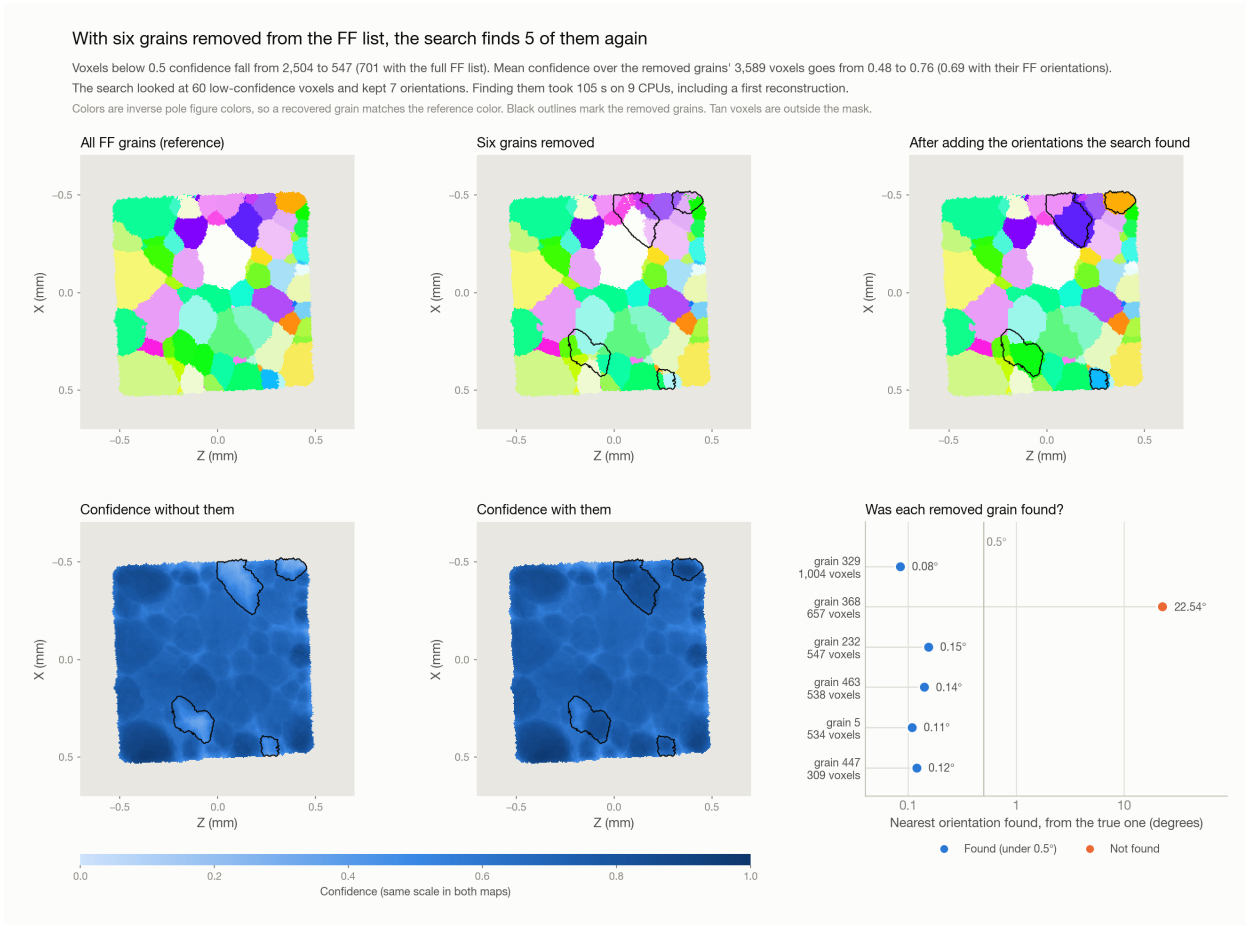


Figure 4: With six grains removed from the FF list, the search finds five of them again. Black outlines mark the removed grains.

5.4 `find_missing_grains.py`

The script has the original search with its bugs fixed, and the new search as the default. `--legacy` selects the original; `--batch-size`, `--grid-spacing` and `--seed` set the new one.

Bugs fixed in the original: the merge of similar orientations treated crystal symmetry wrongly (symmetry operators act on the crystal side of an orientation, $R \cdot S$, and the merge conjugated them, so equivalent orientations were not merged); the brute-force merge and a stall in the loop; and `low_confidence_sparsing` emptied the coordinate list for one-layer reconstructions.

5.5 Dropping orientations that are already known

The search at low-confidence voxels often returns an FF orientation moved by a fraction of a degree. That refines an existing grain, and is not missing from the list. `find_missing_orientations` now drops a result within `known_tol_deg` (default 1°, symmetry aware) of an orientation already in the list; `None` or 0 keeps them, and `n_known` reports how many were dropped.

On the final scan (Figure 5) the search kept 8 orientations, and 5 of them were 0.1 to 0.2°from an FF grain. The other three are 2.4 to 3.4°from FF grains 63, 192 and 360 and explain 697 voxels. Adding only those lowers the voxels below 0.5 confidence from 1,000 to 776 (636 with all eight). There is no reference for the final scan, so these three are candidates to check: they could be separate grains or low-angle variants of the neighboring grains. Their confidence is 0.63 to 0.75, lower than a well-fit grain (about 0.8).

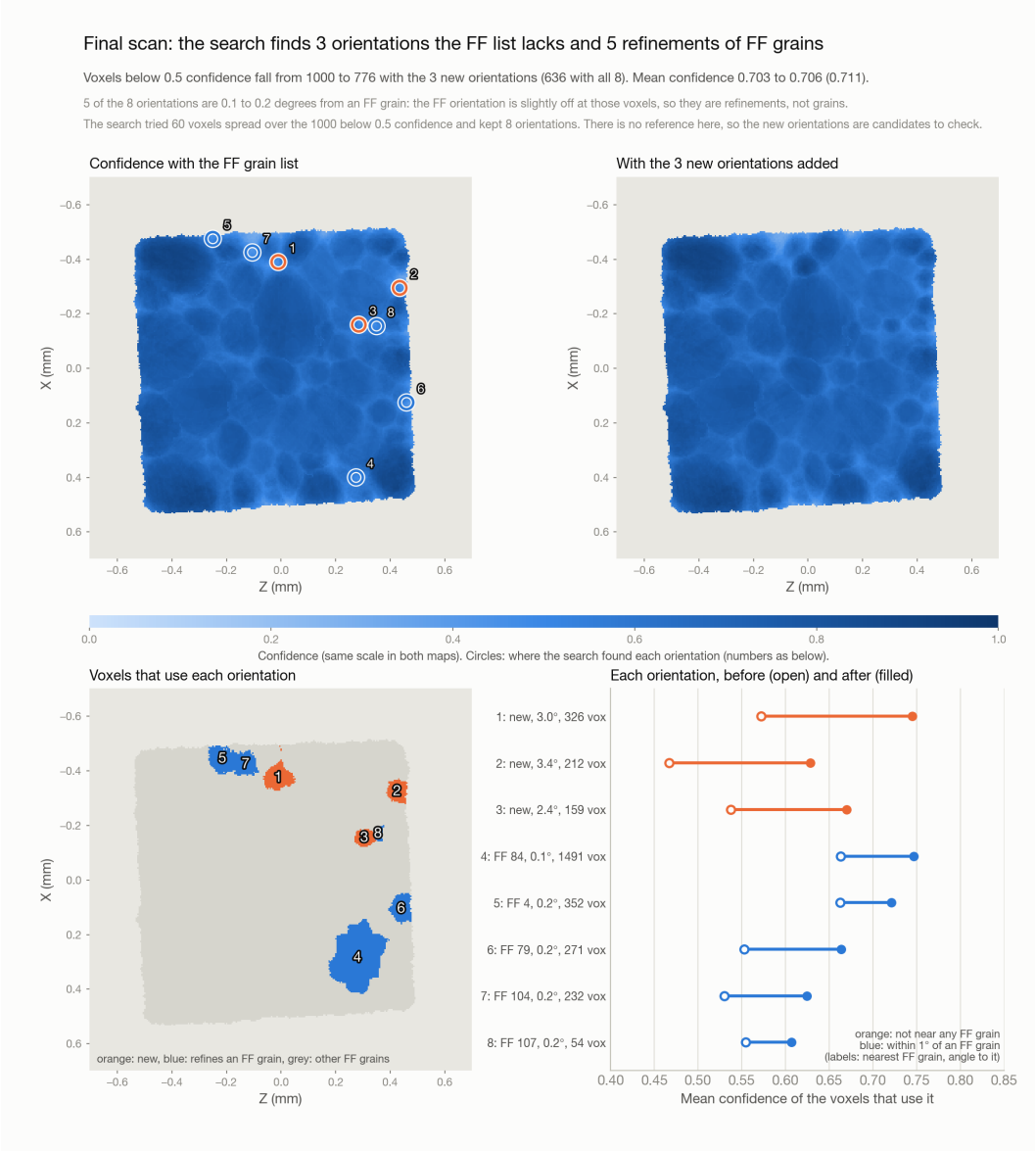


Figure 5: The search on the final scan. Orange: orientations not near any FF grain. Blue: within 1°of an FF grain, which refine it. Numbers match across the panels.

6 Faster orientation refinement, identical results

Refinement moves a voxel's orientation to the one that fits the images best, raising confidence and giving a map of how orientations vary inside grains. Refining one voxel now takes about 0.04 s on one CPU, down from 0.24 s, with exactly the same orientations and confidences (compared bit for bit on 60 voxels).

- 92% of the old time was simulating peaks. The simulation rebuilt the list of symmetry-equivalent reflections on every call and computed pixel sizes nothing used. `simulation.py` (`simulate_angles`) returns exactly the angles `simulateGVecs` does without that, 3 to 4 times faster per orientation, checked identical on 300 random orientations for Ti-7Al (hexagonal) and lowO (cubic).
- The pattern search no longer scores a point twice, because the center of every grid is the current best, whose score is known. That is 314 orientations per voxel instead of 595.

`simulateGVecs` has a quirk that is reproduced exactly: when some reflections have no solution (NaN angles), it keeps the NaN rows and drops the last real peaks. On Ti-7Al this affects 45% of orientations, about 4 of 362 peaks each, and changes a confidence by at most 0.008. It is left as it is so that no score changes. Whether to fix it, in hexrd or in `nfutil`, is open.

7 Refinement along the marching

The marching idea carries over to refinement. A voxel starts from its neighbors' *refined* orientations and needs only a short search, and because each voxel adjusts its neighbor's orientation, the result follows orientation gradients inside a grain. `reconstruct_and_refine_nf.py --spatially-adaptive` (or `nfutil.reconstruct_and_refine_spatially_adaptive`) does it:

1. Seeds are tested against every FF grain and the best is refined with radii 1, 0.5, 0.25, 0.1°.
2. A voxel next to converged voxels scores their refined orientations and refines the best (radii 0.25, 0.1°) if its confidence is at least 0.8 times the best neighbor's.
3. Voxels that fail, or are cut off, are treated like seeds.
4. A boundary pass (below).

Refinement raises the mean confidence of the initial scan from 0.71 to 0.80 and of the final scan from 0.70 to 0.79, and moves orientations by a median of 0.10°(Figure 6). Refined along the marching, the layer takes 92 s against 272 s for refining each voxel alone (88 s against 312 s on the final scan), with the same mean confidence (Figure 7).

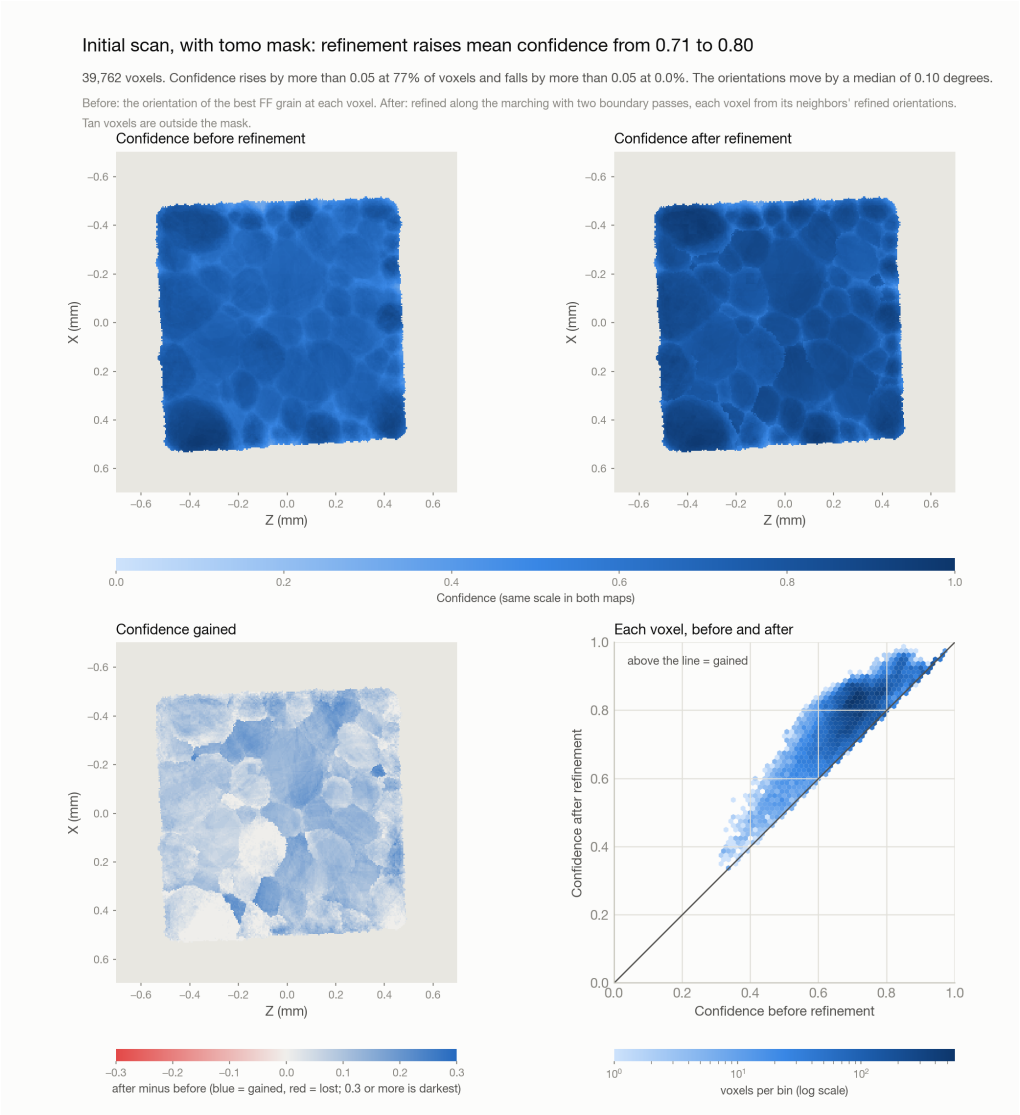


Figure 6: Confidence before and after refinement (initial scan). Confidence rises by more than 0.05 at 77% of voxels and falls by more than 0.05 at almost none.

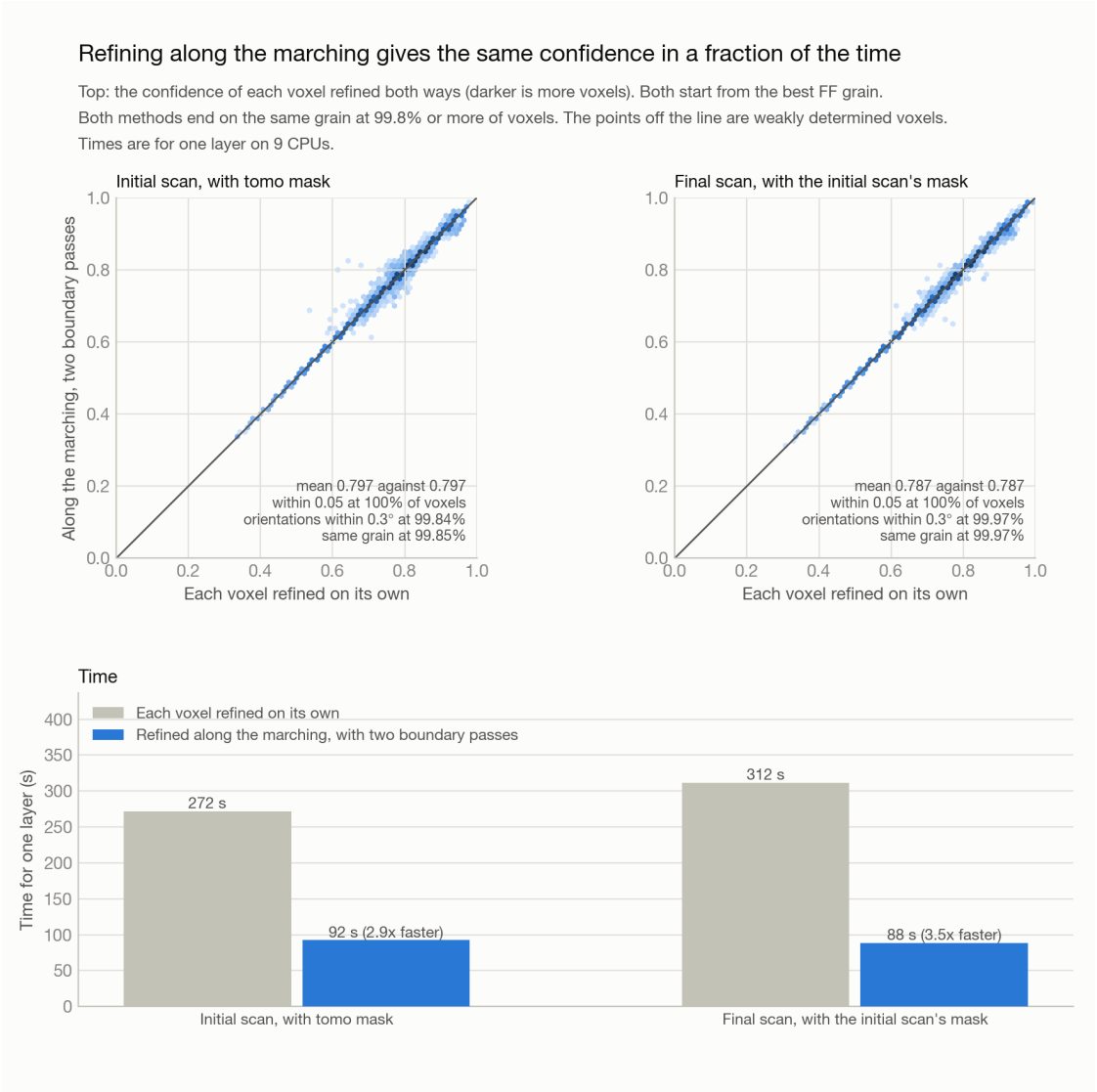


Figure 7: Confidence of each voxel refined along the marching against refined on its own, and the time for one layer.

7.1 The boundary pass

Checking the grain map showed a problem. Boundary voxels are weakly determined, and in the march a grain spreads into them one voxel at a time along the grid axes, so boundaries came out as straight steps with square corners, a few voxels from where they are when each voxel is refined alone. Mean confidence did not show this: the two gave the same mean confidence.

The boundary pass retests the voxels that touch a different grain and those touching them (2 steps) against the grains found within 2 steps of each voxel, and refines the best with the global radii, as independent refinement does. It runs twice (the second pass retests the voxels the first moved to the boundary). The options are `--boundary-width` (2) and `--boundary-passes` (2); 0 turns it off.

	Same grain as refining each voxel alone		Boundary roughness
	Initial scan	Final scan	(straight run, voxels; alone 1.59 / 1.66)
No pass	92.91%	93.54%	2.01 / 2.01
One pass	98.94%	99.22%	1.64 / 1.69
Two passes (default)	99.85%	99.97%	1.58 / 1.66

Table 2: Effect of the boundary pass (initial scan, with the straight run of the boundary in voxels). Testing only the boundary voxels, and not those touching them, leaves 3.0% different after one pass.

The two passes add 30 to 40 s to a 50 s march, and the mean confidence ends the same as refining each voxel alone (0.7972 on the initial scan). A third pass changes 20 voxels.



Figure 8: A 100 by 100 voxel crop of the initial scan, colored by grain id. Without the boundary pass the boundaries are stepped; with two passes they match refining each voxel alone.



Figure 9: The same comparison on the final scan: 99.97% of voxels end on the same grain as refining each voxel alone.

8 Grain segmentation and misorientation from the grain average

A refined field has an orientation at every voxel and no grains until voxels are grouped. `segmentation.py` in `nfutil` defines a grain as a connected set of neighboring voxels whose orientations differ by less than a threshold (default 2°, crystal symmetry counted). The grain average is a symmetry-aware weighted quaternion average, and the misorientation from it of each voxel is the grain reference orientation deviation (GROD).

The threshold hardly matters here. No pair of neighboring voxels differs by 0.5 to 2°, so any threshold from 0.5 to 3°gives identical segments (Figure 10). With 2°the initial scan has 109 segments, 67 of them with 20 or more voxels, and 74 voxels in the others; the final scan has 135 segments, 67 large, with 174 voxels in the others.

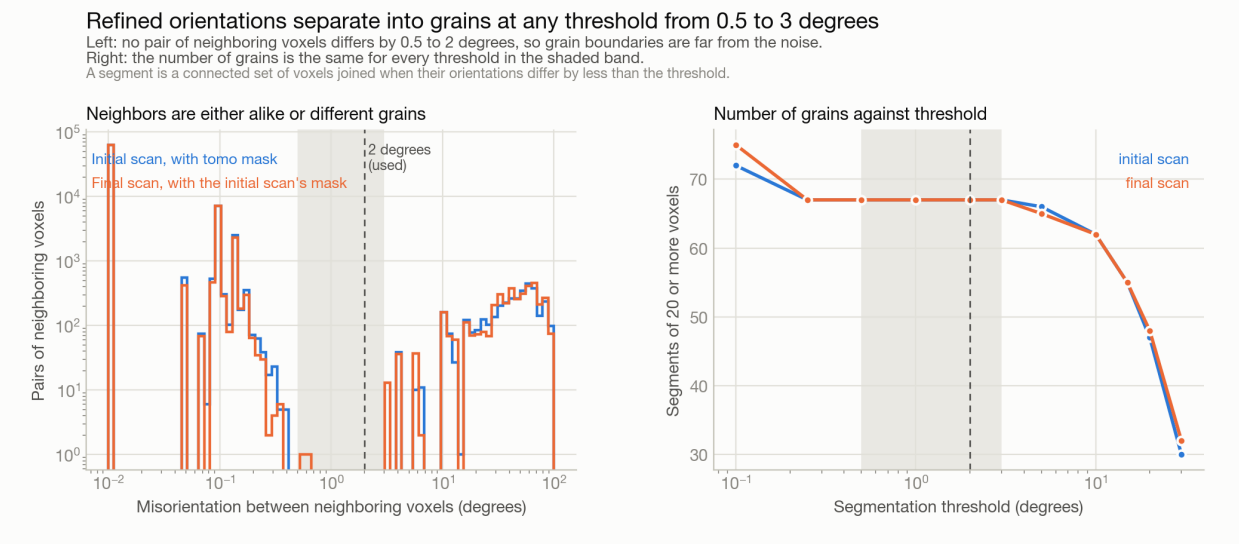


Figure 10: Neighbor misorientation has a gap between 0.5 and 2°(left), and the number of grains is the same for every threshold in the shaded band (right).

The GROD is small: a median of 0.019°on the initial scan (99th percentile 0.15°, largest 0.38°) and 0.022°on the final scan (99th percentile 0.14°, largest 0.69°) (Figures 11 and 12). **This is the spread that the refinement leaves, not measured lattice curvature or strain.** The pattern search moves in steps of 0.25 and 0.1°and a voxel keeps its neighbor's orientation unless a strictly better one is found, so the GROD map is made of flat patches: the 12 largest grains hold between 1 and 34 distinct orientations in total. Seeing real intragranular variation would need a finer final radius, and the images may not be sensitive below about 0.1°.

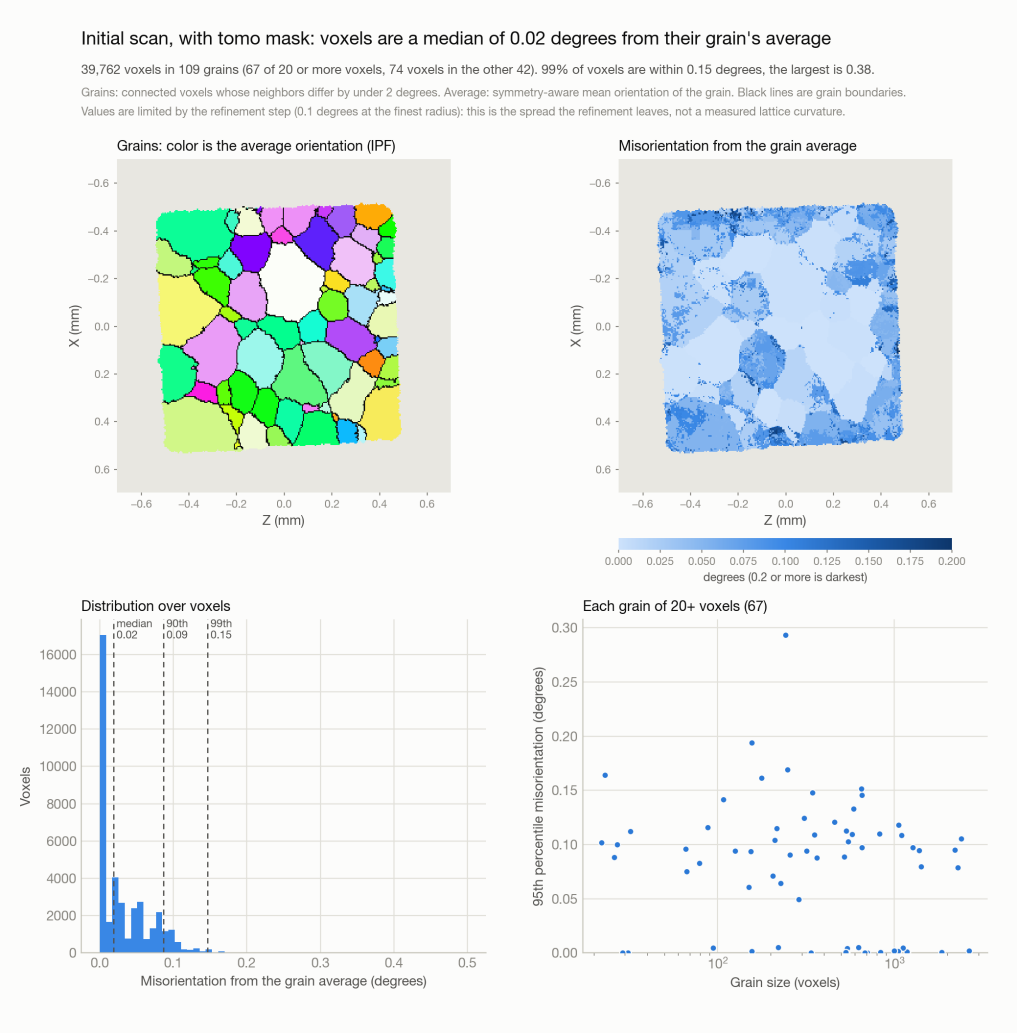


Figure 11: Initial scan. Grains colored by the inverse pole figure color of the grain average, with boundaries; the misorientation of each voxel from the grain average; its distribution; and the 95th percentile for each grain of 20 or more voxels against its size.

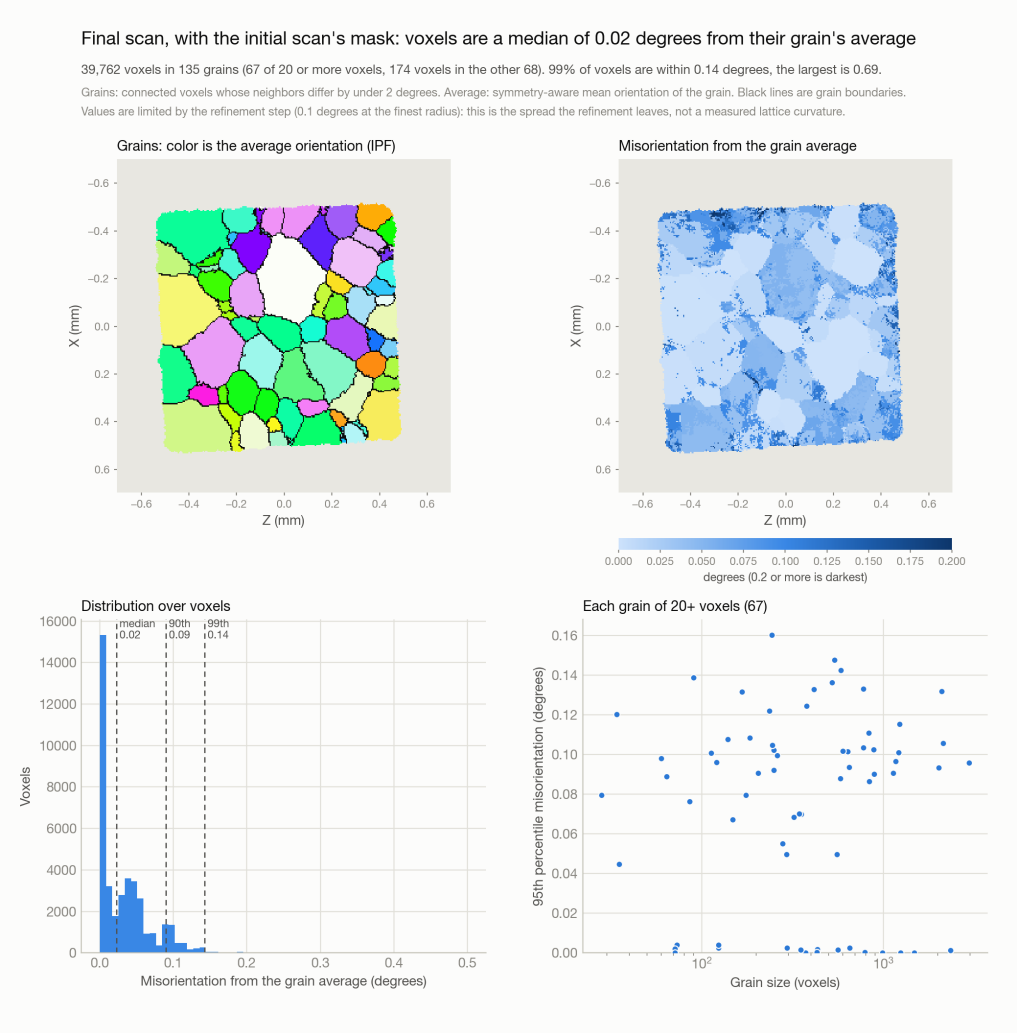


Figure 12: The same for the final scan.

Open questions on the definition: whether grains should follow the FF ids or the orientation-connected regions (they agree to within a grain or two here); whether twins should merge into their parent (at 2°they separate, since they differ by tens of degrees); and whether fragments under 20 voxels should be dropped or folded into a neighbor.

9 Tests

The suite has 81 tests and runs in about 7 minutes. One slow test (leave-out) needs `NF_SLOW_TESTS=1` and was run separately. Tests that need no data cover the chunking, the pattern search, `simulate_angles` against `simulateGVecs`, the marching (a Voronoi phantom with a gradient inside each grain), the boundary band and pass (including restoring boundary voxels the march gave to the wrong grain), the orientation-search helpers, `known_orientations`, and the segmentation (a two-grain phantom with symmetry equivalents). Data tests use Ti-7Al and compare against stored references.

One threshold was changed: the test that the marching simulates fewer orientations per voxel than refining each voxel alone now allows 0.7 of the 314 per voxel (it was 0.6), because the boundary passes add simulations, more on the small unmasked grid.

10 Limitations and open items

- Validated on Ti-7Al, one layer at a time. The local refinement assumes neighbors are already close (refined orientations are about 0.1°from the FF grain here). Faster-varying orientations need larger `local_radii`, and other symmetries may need a different search grid.
- The orientation search finds orientations only. It does not fit position or strain, and a poor result at a voxel can mean a noisy voxel, not a missing grain.
- The boundary pass cannot find a grain that is not within 2 steps of the voxel. Raise `--boundary-width` if boundaries still look stepped.
- A refined orientation is the best fit to the images, not a unique answer: at weak voxels another orientation can fit equally well.
- The unmasked 125 by 125 layer was not rerun after the boundary pass, so its numbers in `orientation_refinement.md` predate it.
- The final-scan search has no reference, so its three new orientations need checking before use as FF grains.
- Open: the `simulateGVecs` NaN-row quirk, the grain definition (twins, small fragments), and a finer final refinement radius to test for real intragranular variation.

Further reading

`find_missing_grains.md` and `orientation_refinement.md` in `nf_hedm` describe the search and the refinement for users of the scripts. Reference: Li and Suter, J. Appl. Cryst. (2013), https://doi.org/10.1107/S0021889813005268.